

A Comparison of Audio, Video, and Multimodal Speaker Identification

GROUP 8: Alexander Baron,
Austin Kleinecke, Dinh Thien Tu Tran

Dataset Overview

VoxCeleb2 Dataset

- 6,112 celebrity speakers
- 150,480 videos from YouTube
- 1,128,246 total utterances / clips
- Pre-split into both AAC files and MP4 files

	A	B	C	D
1	VoxCeleb	VGGFace2	Gender	Set
2	id00012	n000012	m	dev
3	id00015	n000015	m	dev
4	id00016	n000016	m	dev
5	id00017	n000017	m	test
6	id00018	n000018	m	dev
7	id00019	n000019	m	dev
8	id00020	n000020	m	dev
9	id00021	n000021	m	dev
10	id00022	n000022	f	dev

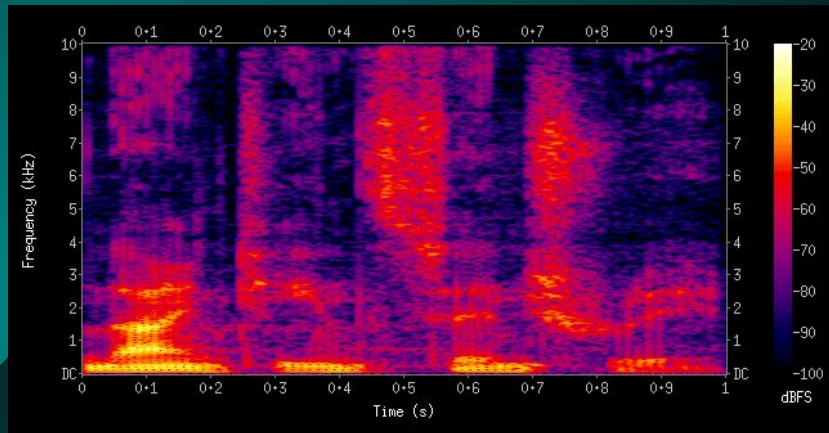
Our created Subset

- 60 speakers
- 20 clips each
- 1,200 paired AAC / MP4 files
- Labels based on spokeed id

	A	B	C	D	E	F	G
1	speaker_i	label	aac_path	mp4_path			
2	id01671		0 aac\id01671	mp4\id01671\7eBNBYBi3dU\00007.mp4			
3	id01671		0 aac\id01671	mp4\id01671\bYHSe6EOEYo\00020.mp4			
4	id01671		0 aac\id01671	mp4\id01671\7eBNBYBi3dU\00005.mp4			
5	id01671		0 aac\id01671	mp4\id01671\75FST6dmbSQ\00002.mp4			
6	id01671		0 aac\id01671	mp4\id01671\o77BaliJL7U\00025.mp4			
7	id01671		0 aac\id01671	mp4\id01671\9Pw5_K3w8P8\00010.mp4			
8	id01671		0 aac\id01671	mp4\id01671\MyTWZ1OfiKw\00013.mp4			
9	id01671		0 aac\id01671	mp4\id01671\7eBNBYBi3dU\00004.mp4			
10	id01671		0 aac\id01671	mp4\id01671\bYHSe6EOEYo\00023.mp4			

Data Preprocessing: Audio

- In order to process the audio it was a choice of using waveforms or spectrograms
- We chose spectrograms since they can be used with simple CNN architectures
- Our dataset consisted of .mp4 files with the speakers
 - We had to convert these .mp4 files into spectrograms
 - Then converting these spectrograms to .npy files for quicker loading



Model Overview: Audio

-First Model

- 2 Layer CNN (1 -> 64) -> (64 -> 128)
- Followed by two fully connected layers

- Second Model

- 3 Layer CNN (1 -> 32) -> (32, 64) -> (64 -> 128)
- Each Convolution was followed by a batch normalization with the corresponding Out_channel
- With max pooling, and dropout applied afterwards
- Finally followed by 2 fully connected layers

- Third Model

- 4 Layer Residual Block (1 -> 64) -> (64 -> 128) -> (128 -> 256) -> (256, 512)
- Each block contains two convolutions (in_channels -> out_channels) -> (out_channels -> out_channels) followed by batch normalization with the out_channel

```
class AudioCNN(torch.nn.Module):
    def __init__(self, num_classes):
        super(AudioCNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm2d(128)

        self.pool = nn.MaxPool2d(2,2)
        self.dropout = nn.Dropout(0.3)

        self.gap = nn.AdaptiveAvgPool2d(1)

        self.fc1 = nn.Linear(128, 256)
        self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
```

```
class AudioResNet(torch.nn.Module):
    def __init__(self, num_classes):
        super(AudioResNet, self).__init__()
        self.layer1 = ResBlock(1, 64, stride=2)
        self.layer2 = ResBlock(64, 128, stride=2)
        self.layer3 = ResBlock(128, 256, stride=2)
        self.layer4 = ResBlock(256, 512, stride=2)

        self.gap = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.gap(x).view(x.size(0), -1)
        x = self.dropout(x)
        return self.fc(x)
```

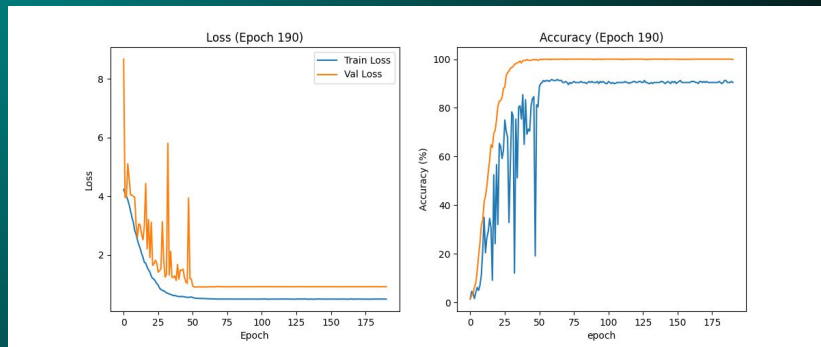
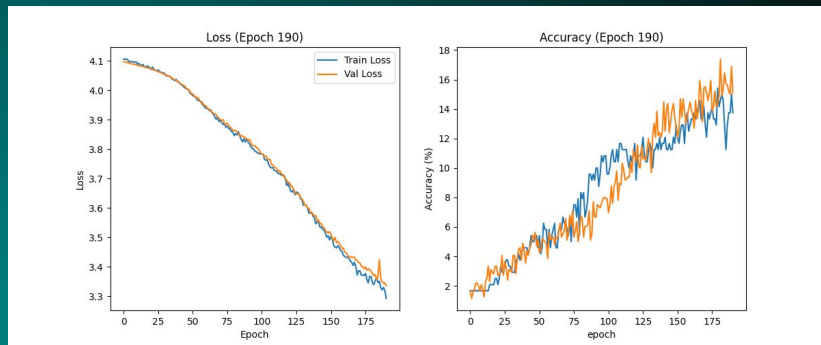
```
class ResBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels: self.shortcut = nn.Sequential(nn.Conv2d(in_channels,
                                                                                               out_channels,
                                                                                               kernel_size=3,
                                                                                               stride=stride,
                                                                                               padding=1))

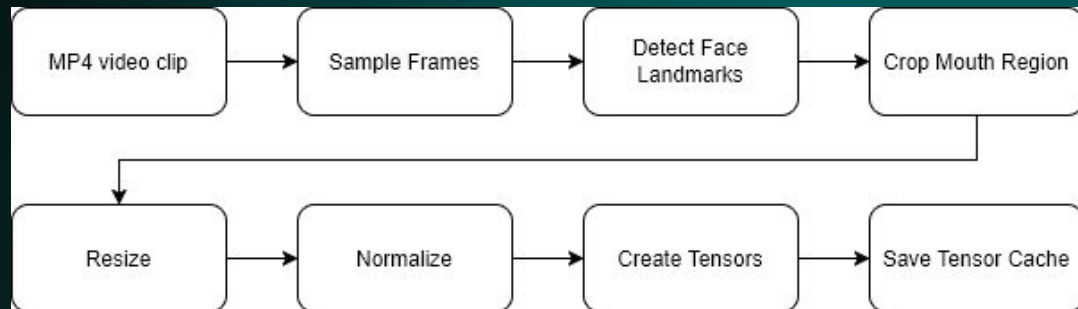
    def forward(self, x):
        identity = self.shortcut(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += identity
        return F.relu(out)
```

Results Overview: Audio

- The first 2 layer CNN got a max accuracy 31%
- Expanding to 3 layers didn't really help at all, it actually took longer to even get back to the previous best of 31%, reaching its best val and train accuracy at epoch 750 with 36%, and 32%
 - Suggesting the model may benefit from more training
- The ResNet gave the best results with a nice 91.25% val accuracy, while getting an unbelievable 100% training accuracy
- Both of the simple cnn's had a learning rate set to 0.00003, the ResNet was set to 0.001. All three models used the Adam optimizer



Data Preprocessing: Video



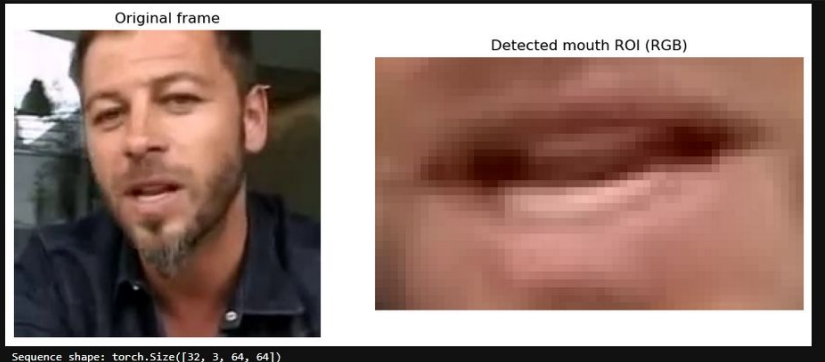
- 1,200 total sample clips
 - (60 speakers x 20 clips each)
- 80 / 20 split
 - 960 training clips, 240 validation clips
 - 60 classes
- Each sample becomes 32 mouth-roi frames
- Final model input tensor: [32, 3, 64, 64]

```
face_landmarker = Facelandmarker.create_from_options(landmarker_options)

# Example: test a single sample before training
sample_row = rows[0]
sample_video = DATA_ROOT / fix_path(sample_row["mp4_path"])

show_mouth_debug(sample_video)

frames = extract_roi_sequence(sample_video, num_frames=NUM_FRAMES, img_size=IMG_SIZE)
print("Sequence shape:", frames.shape) # Expect [T, 3, 64, 64]
```



Sequence shape: torch.Size([32, 3, 64, 64])

Model Overview: Video

Final CNN-GRU Model:

- **Input:** 32 RGB mouth crops
 - [B, T, 3, H, W]
- **Frame encoder:** CNN extracts mouth-shape features from each mouth crop
- **GRU sequence model:** Learns mouth motion across time
- **Classifier:** Maps the clip summary to 60 speaker classes

```
class MouthVideoCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()

        # Extract features from one frame at a time
        self.frame_encoder = nn.Sequential(
            # Input = [3, 64, 64]
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.LeakyReLU(0.1),
            # [32, 32, 32]
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1),
            # [64, 16, 16]
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.1),
            # Reduce each map to a single value
            # [128, 1, 1]
            nn.AdaptiveMaxPool2d((1, 1)),
        )

        # Reads frame features in order and builds memory of the
        # sequence allowing the model to learn motion
        self.gru = nn.GRU(
            input_size=128,
            hidden_size=256,
            batch_first=True,
            bidirectional=True
        )

        self.dropout = nn.Dropout(0.4)

        # Final classifier layer
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        """
        Input shape: x = [B, T, 3, H, W]
        B = batch size
        T = num of frames
        3 = RGB channel
        H, W = img size
        """
        # Save original shape pieces
        b, t, c, h, w = x.shape

        # Combine batch and time so the CNN can process frames individually
        x = x.view(b * t, c, h, w)

        # Encode every frame
        x = self.frame_encoder(x)

        # Remove the last two size-1 dimensions
        x = x.view(b, t, 128)

        # Pass sequence of frame features into GRU
        # _, h_n = self.gru(x)
        gru_out, h_n = self.gru(x)

        # h_n contains the final hidden state (summary of sequence)
        # x = h_n[-1]
        x = gru_out.mean(dim=1)

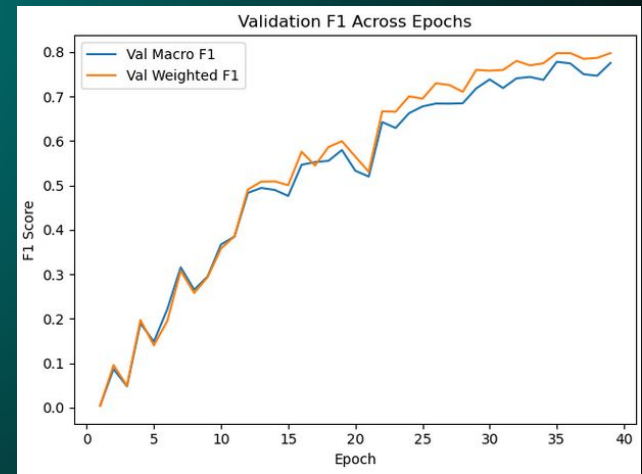
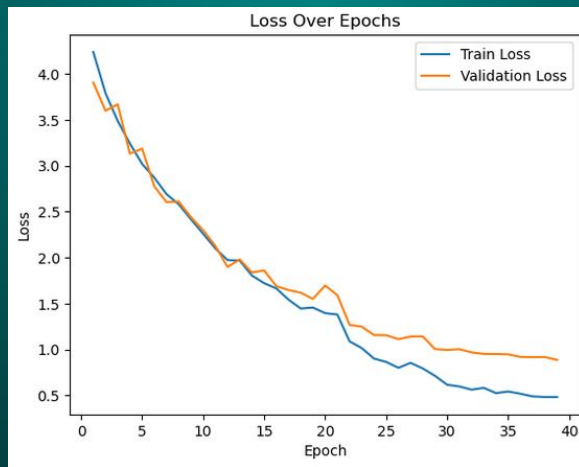
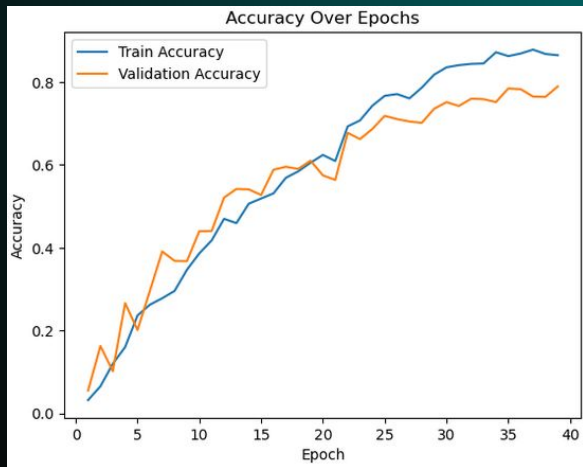
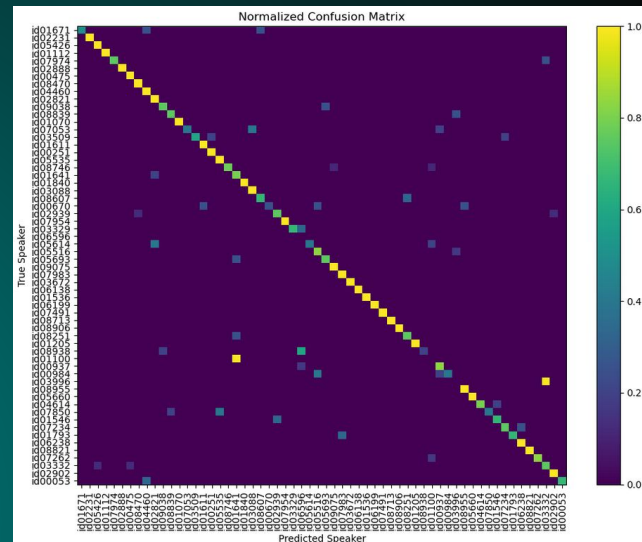
        # Run dropout layer
        x = self.dropout(x)

        # Classify
        x = self.fc(x)

        return x
```

Results Overview: Video

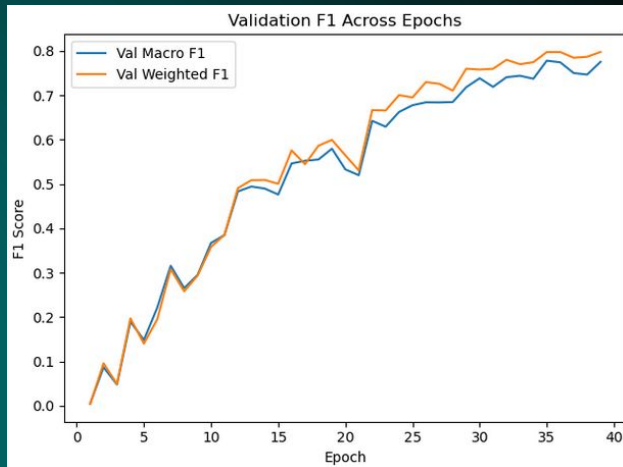
- **Best epoch: 39 / 40**
 - **Best Training Accuracy: 86.45%**
 - **Best Validation Accuracy: 78.93%**
 - **Best Macro F1 Score: 77.52%**
-
- **Best Training Loss: 0.4833**
 - **Best Validation Loss: 0.8868**



Baseline Comparison: Video to ResNet-18

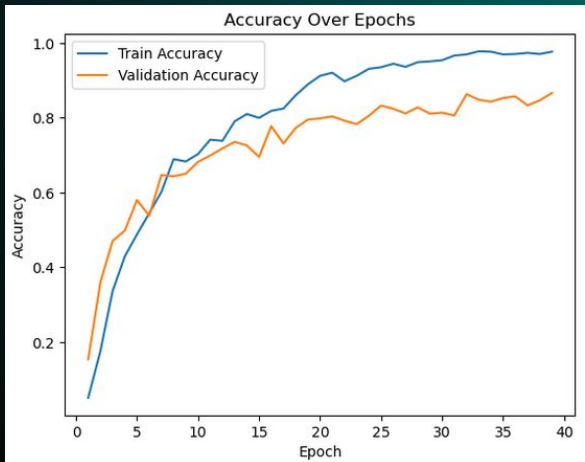
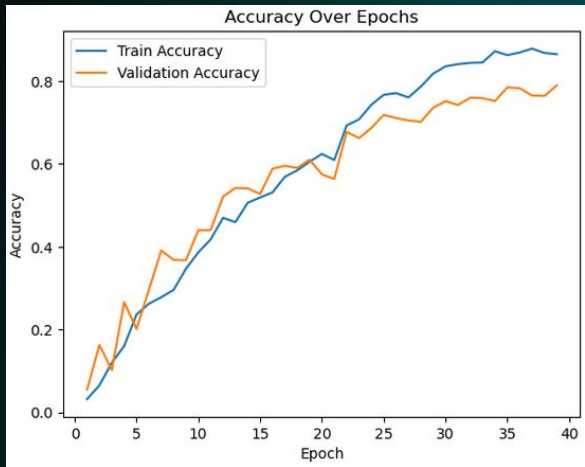
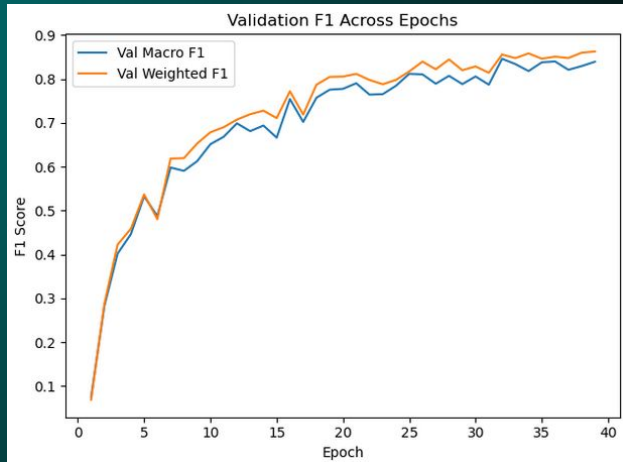
Custom 3-layer CNN-GRU (top):

- 78.93% Validation Accuracy
- 77.53% Macro F1
- 79.75% Weighted F1



Pretrained ResNet-18 (bottom):

- 86.66% Validation Accuracy
- 82.53% Macro F1
- 86.27% Weighted F1



Model Overview: Multimodal

- Two frozen backbone
- Both feature vectors concatenated:
1024-dim vector
- The only part that got trained is the Fusion Head

```
VIDEO_FEAT_DIM = 512
AUDIO_FEAT_DIM = 512
FUSED_DIM       = VIDEO_FEAT_DIM + AUDIO_FEAT_DIM
```

```
class MultimodalFusionModel(nn.Module):
    def __init__(self, video_model, audio_model, num_classes,
                 freeze_backbones=True):
        super().__init__()
        self.video_model = video_model
        self.audio_model = audio_model

        if freeze_backbones:
            for p in self.video_model.parameters():
                p.requires_grad = False
            for p in self.audio_model.parameters():
                p.requires_grad = False

        self.fusion_head = nn.Sequential(
            nn.Linear(FUSED_DIM, 512),
            nn.BatchNorm1d(512),
            nn.LeakyReLU(0.1),
            nn.Dropout(0.4),

            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.LeakyReLU(0.1),
            nn.Dropout(0.3),

            nn.Linear(256, num_classes),
        )

    def forward(self, frames, spectrogram):
        v = self.video_model.extract_features(frames)
        a = self.audio_model.extract_features(spectrogram)
        fused = torch.cat([v, a], dim=1)
        return self.fusion_head(fused)
```

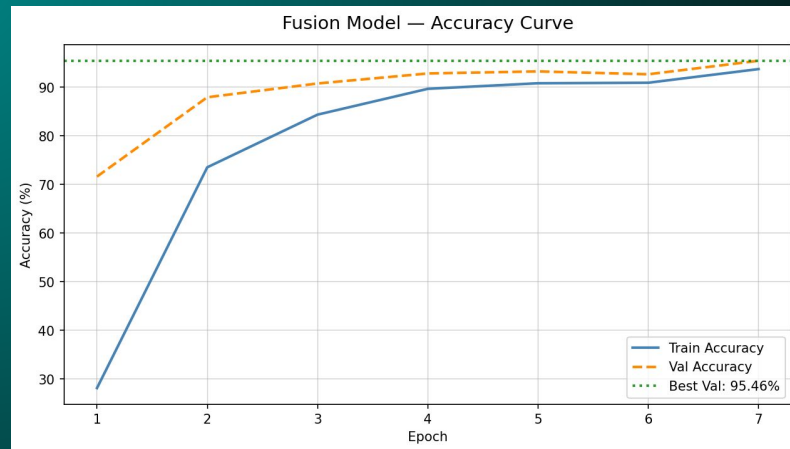
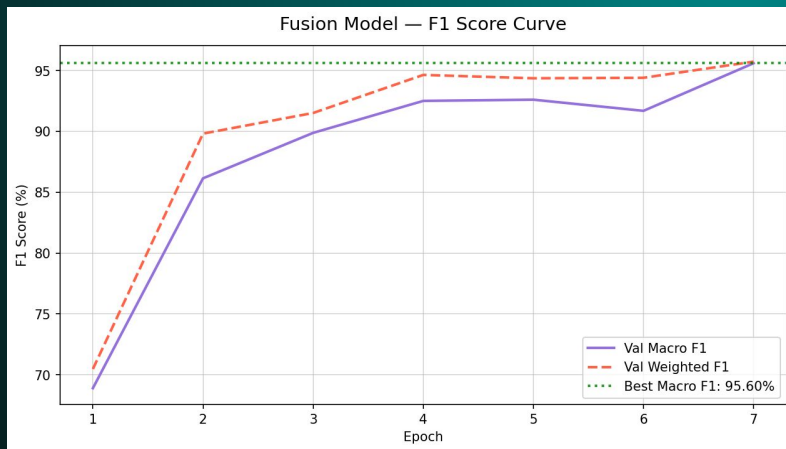
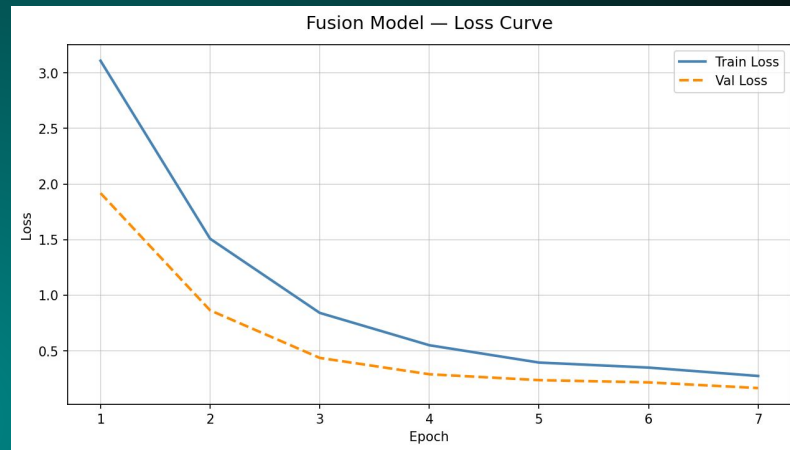
Results Overview: Multimodal

Best Epoch: 7/30

Best Val Acc: 95.4614%

Best Val Loss: 0.165

Best Macro F1: 95.6%



Conclusion

Audio Model:

- The audio branch showed that scaling up does not yield better results than a smarter architecture
 - For the CNN scaling up actually started to hurt performance after 3 layers
- All three models were sensitive to their learning rates
- All three models ran surprisingly quick with the ResNet being the quickest, taking ~1.2 seconds per epoch

Video Model:

- The video branch confirmed that mouth movement can be a useful speaker-identification signal.
- Standardizing clips into fixed-length mouth sequences made the model input consistent and trainable.
- A stronger encoder improved final results, but the overall CNN/ResNet-to-GRU pipeline was the key design pattern.

Multimodal model:

- Overperforms both unimodal approaches
- Multimodal fusion is robust — audio and video capture different aspects of speaker identity; when combined, they compensate for each other's weaknesses, validating our hypothesis that cross-modal redundancy improves classification accuracy